

Question No:10

Problem Description

Mathison has uncovered an ancient piece of paper on which are written N integers. This set of numbers will be referred to as A . Mathison learned in History class that a group of integers is unique if and only if their total is divisible by a mythological constant M .

Mathison tries to find out how many **distinct** triplets of numbers, from the piece of paper, have their sum divisible by M . Unfortunately, this problem is quite hard to crack and he needs your help.

Constraints

- $1 \leq N \leq 2 \times 10^5$
- $1 \leq M \leq 10^4$
- $0 \leq A[i] \leq 2 \times 10^8$

Input

The first line of the input file contains two space-separated integers, N and M , representing the number of integers and the mythical constant.

The next line contains N space-separated integers, where the i^{th} integer represents $A[i]$.

Output

The output file should contain only one integer, the answer to Mathison's question.

Explanation for Test case 1

There are 26 special trios: {0, 1, 2}; {0, 1, 8}; {0, 2, 5}; {0, 2, 6}; {0, 2, 9}; {0, 3, 7}; {0, 5, 8}; {0, 6, 8}; {0, 8, 9}; {1, 2, 7}; {1, 3, 4}; {1, 5, 6}; {1, 5, 9}; {1, 6, 9}; {1, 7, 8}; {2, 4, 8}; {2, 5, 7}; {2, 6, 7}; {2, 7, 9}; {3, 4, 5}; {3, 4, 6}; {3, 4, 9}; {5, 6, 9}; {5, 7, 8}; {6, 7, 8}; {7, 8, 9};

Note: Here we only show the **positions** (0-indexed) in each trio.

Test Case 1	Test Case 2
INPUT (STDIN)	INPUT (STDIN)
10 5	10 5
1 10 4 3 2 5 0 1 9 5	11 10 14 31 21 15 10 11 9 51
EXPECTED OUTPUT	EXPECTED OUTPUT
26	31

Code:

```
1  #include <stdio.h>
2
3  int main() {
4      int N, M;
5      scanf("%d %d", &N, &M);
6
7      int freq[10001] = {0};
8
9      for (int i = 0; i < N; i++) {
10         int x;
11         scanf("%d", &x);
12         freq[x % M]++;
13     }
14
15     long long ans = 0;
16
17     // Choose remainders r <= s <= t
18     for (int r = 0; r < M; r++) {
19         for (int s = r; s < M; s++) {
20
21             int t = (M - (r + s) % M) % M;
22
23             // Count each combination only once
24             if (t < s) {
25                 continue;
26             }
27
28             long long a = freq[r];
29             long long b = freq[s];
```

```

30     long long c = freq[t];
31
32     if (r == s && s == t) {
33         // All three remainders are equal
34         if (a >= 3) {
35             ans += a * (a - 1) * (a - 2) / 6;
36         }
37     }
38     else if (r == s) {
39         // First two remainders are equal
40         ans += a * (a - 1) / 2 * c;
41     }
42     else if (s == t) {
43         // Last two remainders are equal
44         ans += a * b * (b - 1) / 2;
45     }
46     else {
47         // All three remainders are different
48         ans += a * b * c;
49     }
50 }
51
52
53 printf("%lld\n", ans);
54
55 return 0;
56 }

```

Output:

Test Case-1:

```

10 5
1 10 4 3 2 5 0 1 9 5
26
PS C:\Users\kesha\OneDrive\Documents\3s

```

Test Case-2:

```

10 5
11 10 14 31 25 10 11 9 5 1
36
PS C:\Users\kesha\OneDrive\Documents\3sem\DSA\

```

Keshav Kumar Rauniayar

CH.SC.U4CSE25236

